Digital Signal Processing for Sound and Music

Integrated lecture: welcome, course map, and mathematical foundations

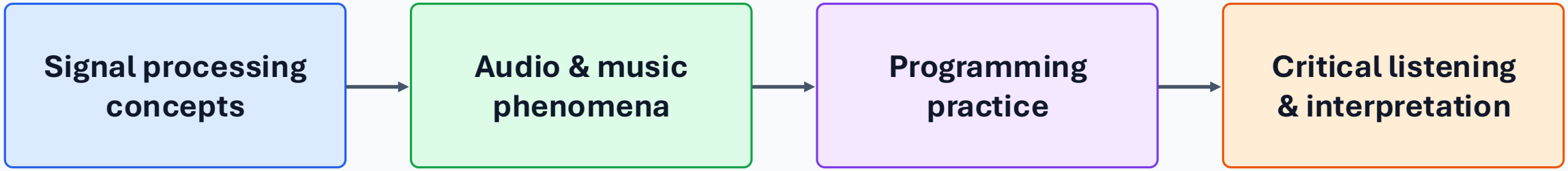
Digital Signal Processing for Sound and Music · Lecture 1

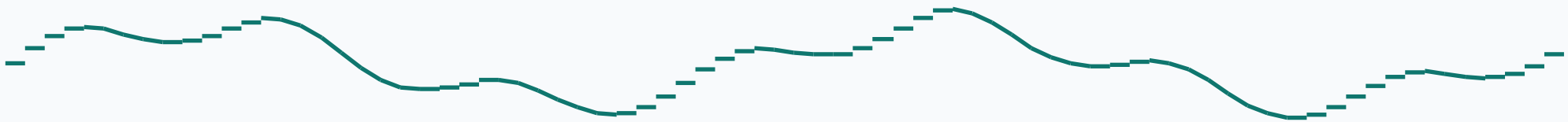
INTRO

Xavier Serra · Universitat Pompeu Fabra

Why this course?

The goal is not only to know formulas, but to use them to listen, analyze, transform, and understand musical sounds.





Who is the course for?

Students interested in musical sound as both a physical signal and a creative medium.

- Comfort following mathematical notation: sequences, complex numbers, dot products, convolution.
- Willingness to program, experiment, debug, and listen critically.
- Interest in analysis, synthesis, transformations, and computational descriptions of sound.

Course habit: every concept should become audible, computable, and inspectable.

MATERIALS

Open course materials

The course uses the MTG sms-tools-materials repository as the common space for lectures, examples, exercises, sounds, and GUI interfaces.

lectures

examples notebooks

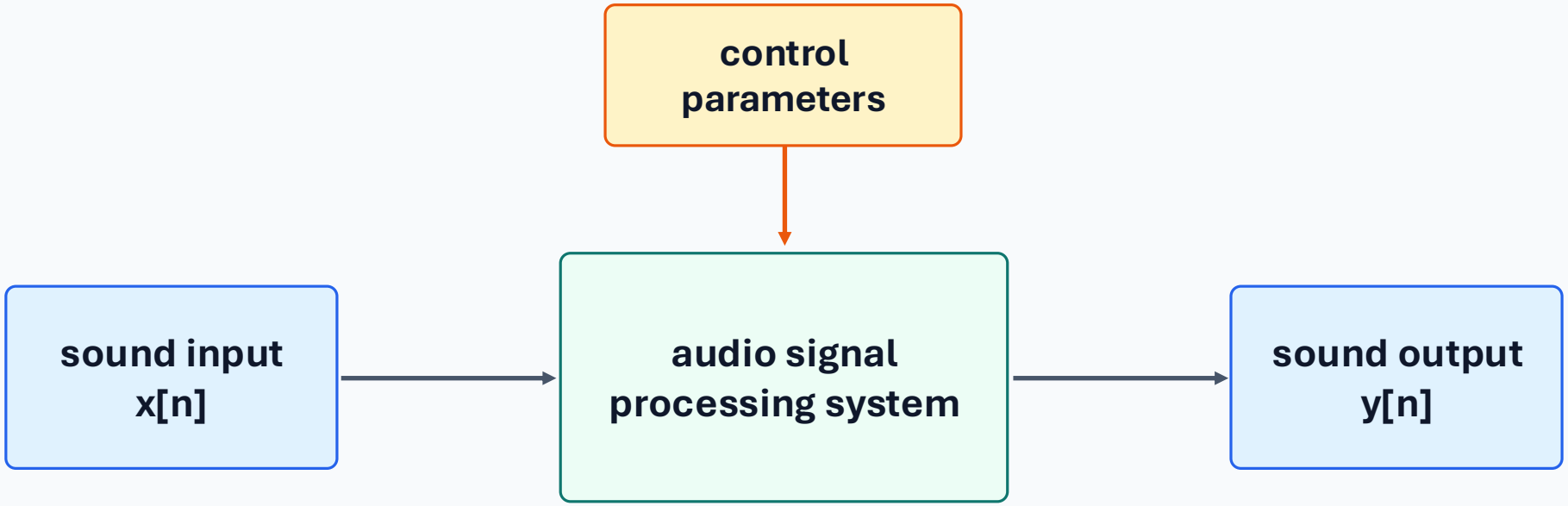
exercises

sounds

GUI tools

- Use notebooks for reproducible DSP experiments.
- Use GUIs to connect parameters, plots, and listening.
- Use your own audio only after checking the method on known examples.

What is audio signal processing?

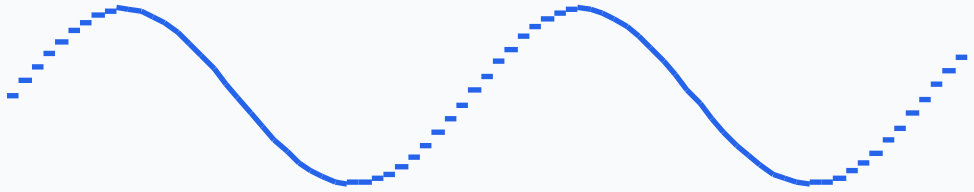


Intentional alteration, analysis, representation, or generation of sound signals.

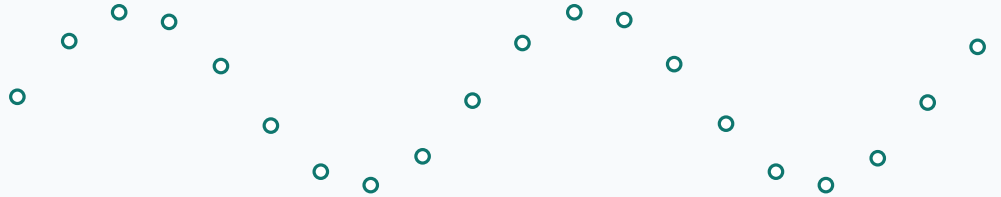
Analog and digital signals

Analog signals vary continuously in time and amplitude; digital audio represents them as a sequence of numbers.

Continuous waveform



Sampled waveform



- Sampling rate fixes time resolution: e.g., 44.1 kHz means 44,100 samples per second.
- Quantization fixes amplitude resolution: each sample is stored with finite precision.

Core application areas

Storage Represent sound files and streams reliably.

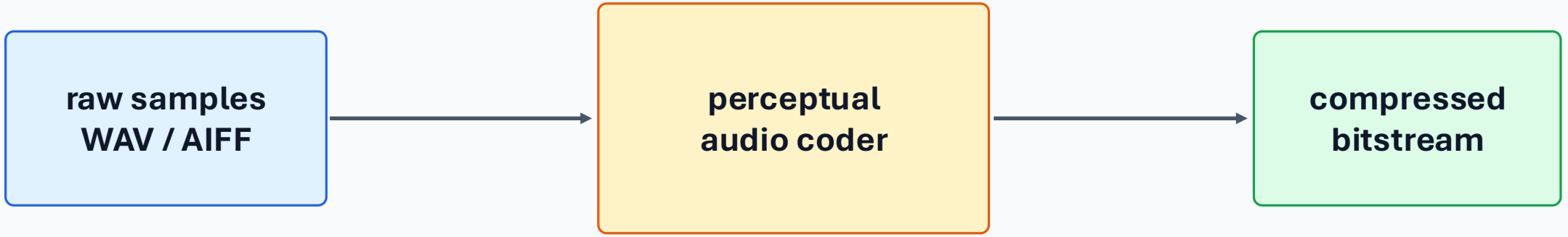
Compression Use signal structure and perception to reduce data.

Effects Alter timbre, space, pitch, or timing.

Synthesis Generate sound from models and parameters.

Description Extract features for search, MIR, and machine learning.

From storage to compression



Compression is not just “smaller files”: it is a compromise between bit-rate, perceived quality, latency, and compatibility.

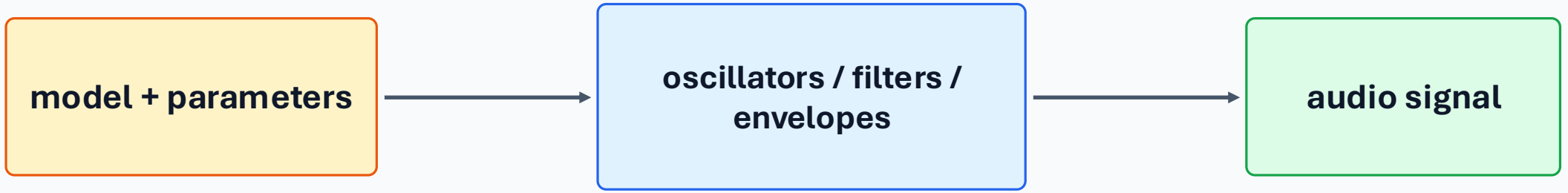
APPLICATIONS

Transformations and effects

A transformation changes how a sound is perceived while preserving or reshaping selected musical attributes.

- filtering / EQ
- delay / echo
- reverberation
- chorus / flanger
- pitch shifting
- time stretching
- morphing
- spatial audio

Synthesis: parameters become sound



- Additive: sum of sinusoids.
- Subtractive: filter a rich source.
- FM: modulate oscillator frequency.
- Sampling, granular, physical, and spectral methods.

Description: measurements of sound and music

Audio features connect signals to musical concepts, but every descriptor has assumptions and limitations.

OUTLINE

Course structure: 10 topics

1 Intro + math

2 DFT

3 Fourier properties

4 STFT

5 Sinusoidal model

6 Harmonic model

7 Sinusoidal + residual

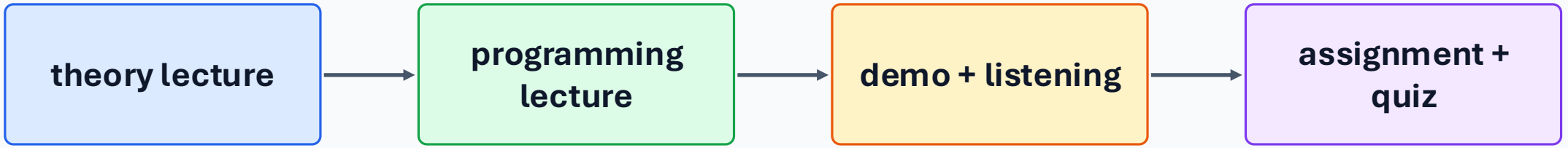
8 Transformations

9 Sound/music description

10 Concluding topics

The sequence moves from mathematical representation to analysis/synthesis models, transformations, and descriptions.

Learning workflow every week



- Start from a signal-processing idea.
- Implement a minimal example.
- Inspect time-domain and spectral plots.
- Listen to the result and interpret what changed.

MATH

Mathematics: why these tools?

The DFT, STFT, and spectral models are built from a compact set of mathematical ideas.

- sinusoids
- complex numbers
- Euler formula
- complex sinusoids
- dot products
- orthogonality
- even/odd symmetry
- convolution

MATH

Sinusoidal functions

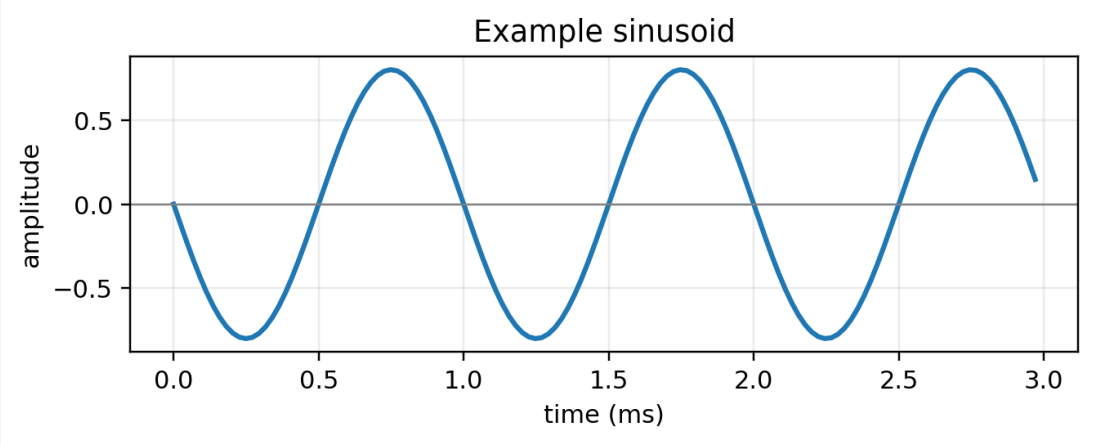
Continuous-time signal

$$x(t) = A\cos(2\pi f_0 t + \phi)$$

Discrete-time signal

$$x[n] = A\cos(2\pi f_0 n / f_s + \phi)$$

A: amplitude
f_0: frequency in Hz
ϕ: phase shift
f_s: sampling rate, which maps time into sample index n



A sinusoid is fully described by amplitude, frequency, and phase. In digital audio we work with the sampled version x[n].

MATH

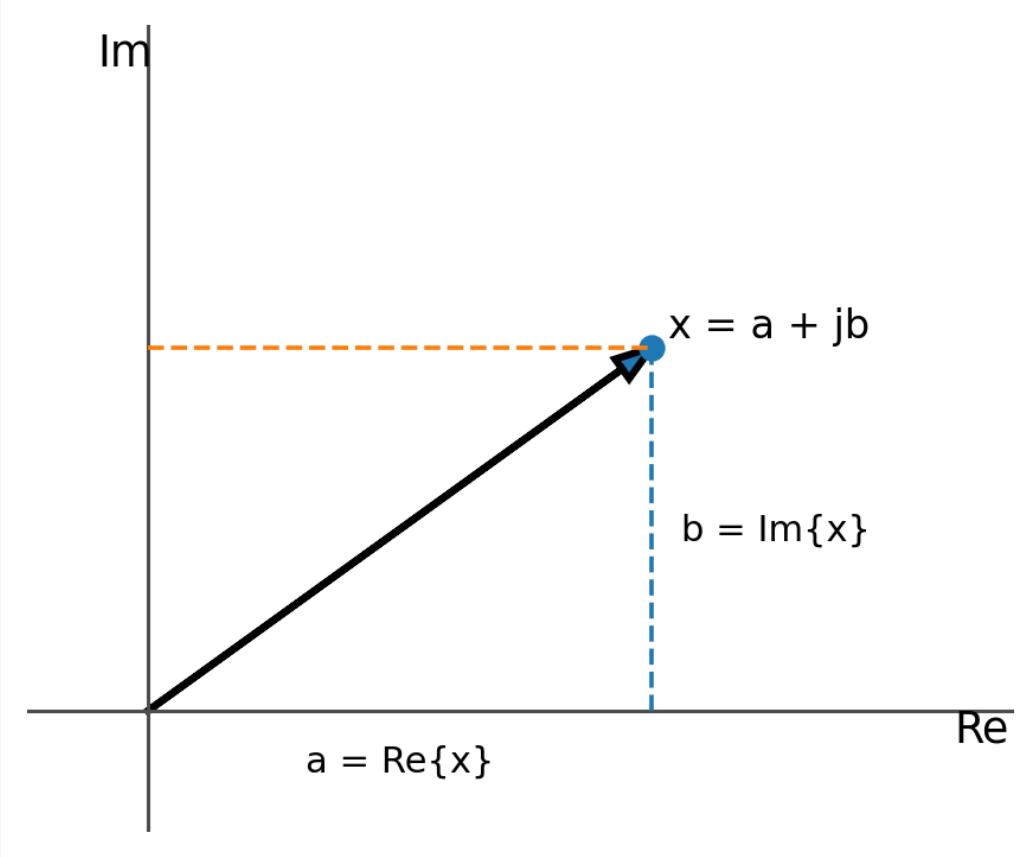
Complex numbers and the complex plane

$$x = a + jb$$

$$\mathrm{Re}\{x\} = a, \quad \mathrm{Im}\{x\} = b, \quad j^2 = -1$$

A complex number is represented as a point or vector on the complex plane.

The horizontal axis is real; the vertical axis is imaginary.



MATH

Rectangular and polar forms

Rectangular

$$x = a + jb$$

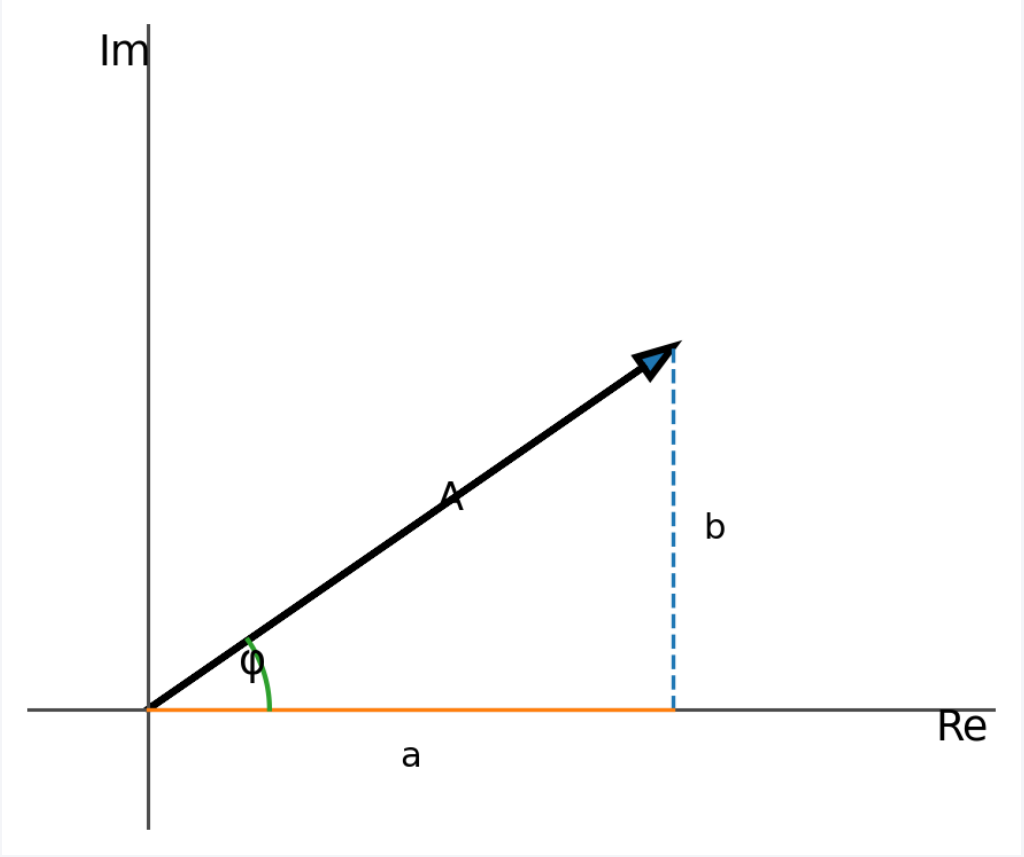
Good for addition and subtraction: real and imaginary parts are explicit.

Polar

$$x = Ae^{j\phi}$$

$$A = |x| = \sqrt{a^2 + b^2} \qquad \phi = \arg(x) = \operatorname{atan2}(b, a)$$

Good for magnitude, phase, multiplication, and rotation. This is the form used to describe spectral magnitude and phase.



MATH

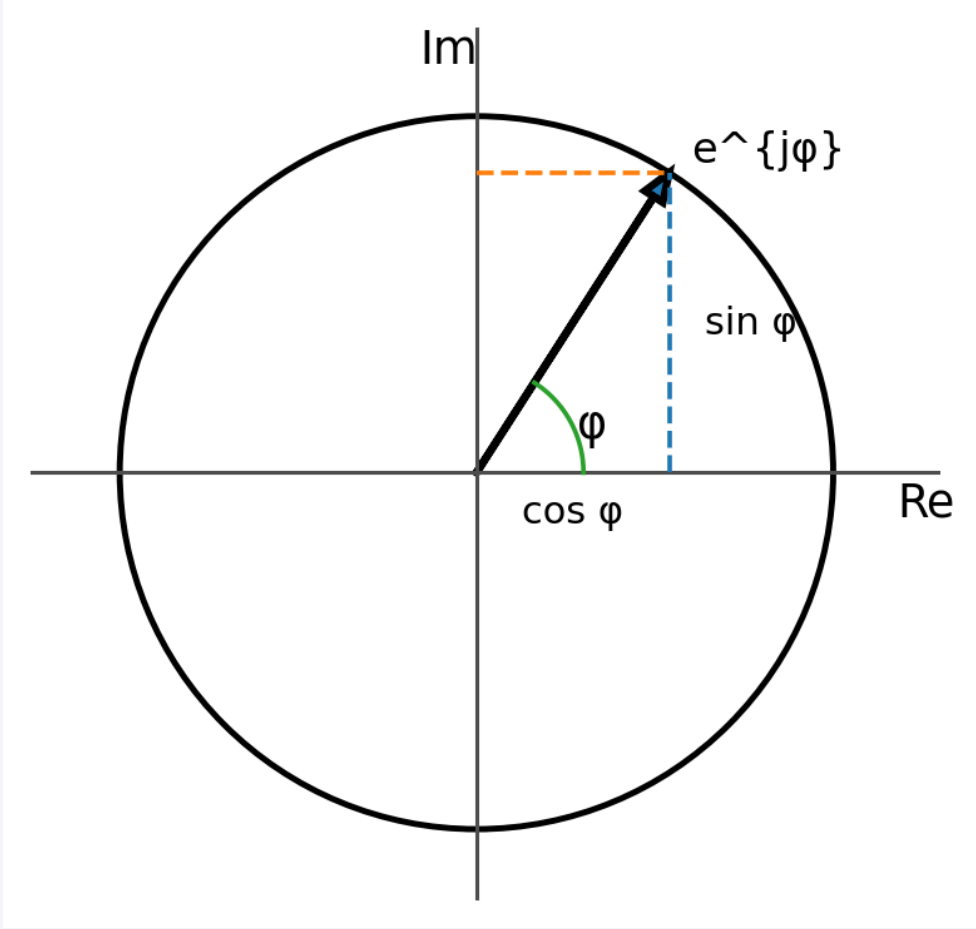
Euler’s formula: rotation and sinusoids

$$e^{j\phi} = \cos(\phi) + j\sin(\phi)$$

$$\cos\phi = \frac{e^{j\phi} + e^{-j\phi}}{2}$$

$$\sin\phi = \frac{e^{j\phi} - e^{-j\phi}}{2j}$$

A complex exponential is a rotating point. Its real and imaginary projections are cosine and sine.



MATH

Real and complex sinusoids

Complex sinusoid

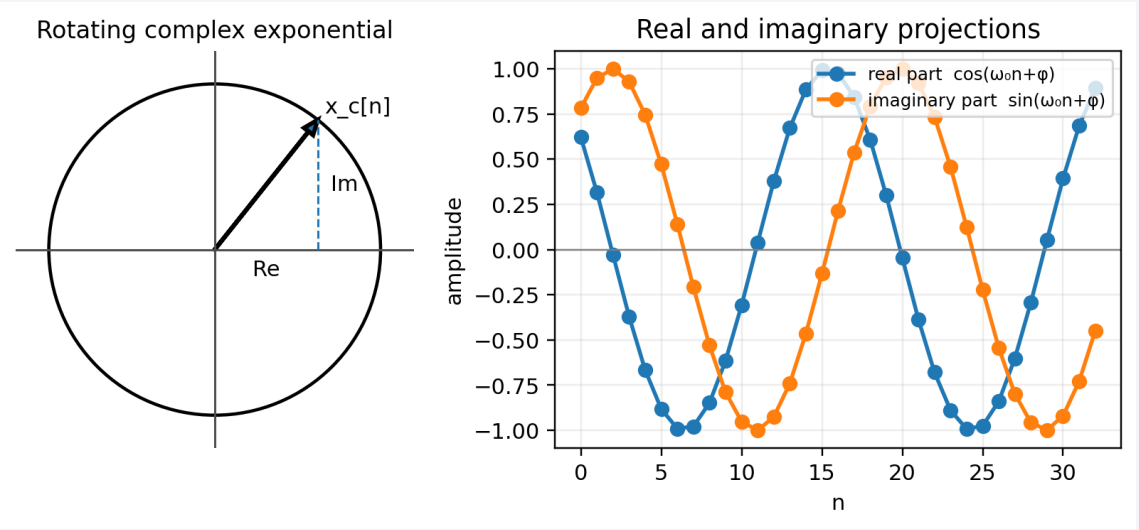
$$\bar{x}[n] = Ae^{j(\omega nT + \phi)} = Ae^{j\phi}e^{j\omega nT} = Xe^{j\omega nT}$$

$$\bar{x}[n] = A\cos(\omega nT + \phi) + jA\sin(\omega nT + \phi)$$

Real sinusoid

$$x[n] = A\cos(\omega nT + \phi) = A\left(\frac{e^{j(\omega nT + \phi)} + e^{-j(\omega nT + \phi)}}{2}\right)$$

$$= \frac{1}{2}Xe^{j\omega nT} + \frac{1}{2}X^{*}e^{-j\omega nT} = \frac{1}{2}\bar{x}[n] + \frac{1}{2}\bar{x}^{*}[n] = \Re\{\bar{x}[n]\}$$



The real signal is the real part of the complex sinusoid; the imaginary part is the quadrature projection.

MATH

Scalar product of sequences

Definition

$$\langle x, y \rangle = \Sigma\, x[n]\, y^*[n]$$

sum over n = 0, …, N−1

The scalar product measures similarity between two sequences.

Example with complex-valued sequences

$$x[n] = [0, j, 1] \quad y[n] = [1, j, j]$$

$$y^*[n] = [1, -j, -j]$$

$$\langle x, y \rangle = 0 \cdot 1 + j \cdot (-j) + 1 \cdot (-j) = 1 - j$$

For complex sequences, the second sequence is conjugated before multiplication.

Element-by-element products

n	0	1	2
x[n]	0	j	1
y*[n]	1	−j	−j
x[n] y*[n]	0	1	−j

$$\text{sum} = 0 + 1 - j = 1 - j$$

MATH

Orthogonality of sequences

Definition

$$x \perp y \Leftrightarrow \langle x, y \rangle = 0$$

Orthogonal sequences have zero scalar product.

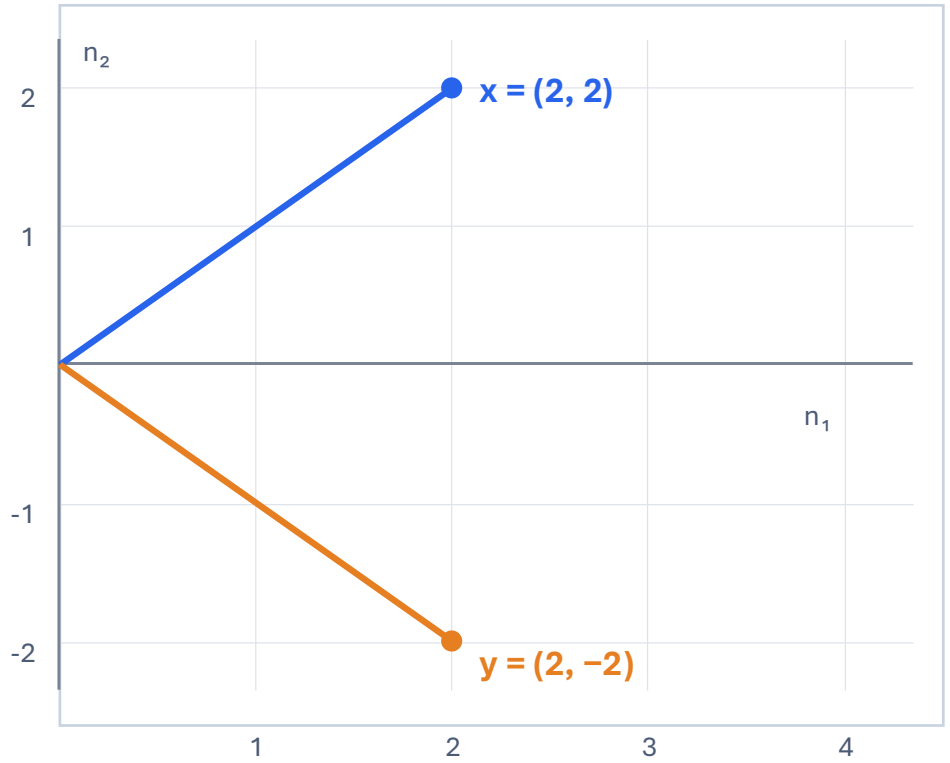
Simple geometric example

$$x = (2, 2) \quad y = (2, -2)$$

$$\langle x, y \rangle = 2 \cdot 2 + 2 \cdot (-2) = 0$$

The two vectors are perpendicular. This is the geometric intuition behind orthogonal basis functions in the DFT.

Vector view of the example



Because ⟨x, y⟩ = 0, the two directions do not “share” any component.

MATH

Even and odd functions

Definitions

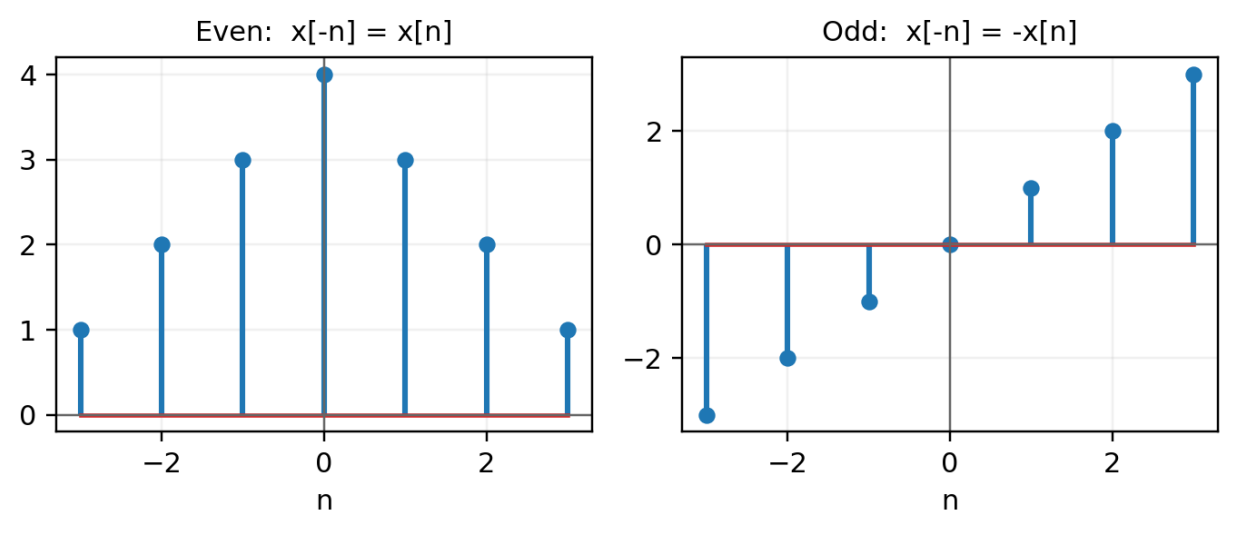
Even: $x[-n] = x[n]$

Odd: $x[-n] = -x[n]$

Decomposition

$$x_e[n] = \frac{x[n] + x[-n]}{2} \qquad x_o[n] = \frac{x[n] - x[-n]}{2}$$

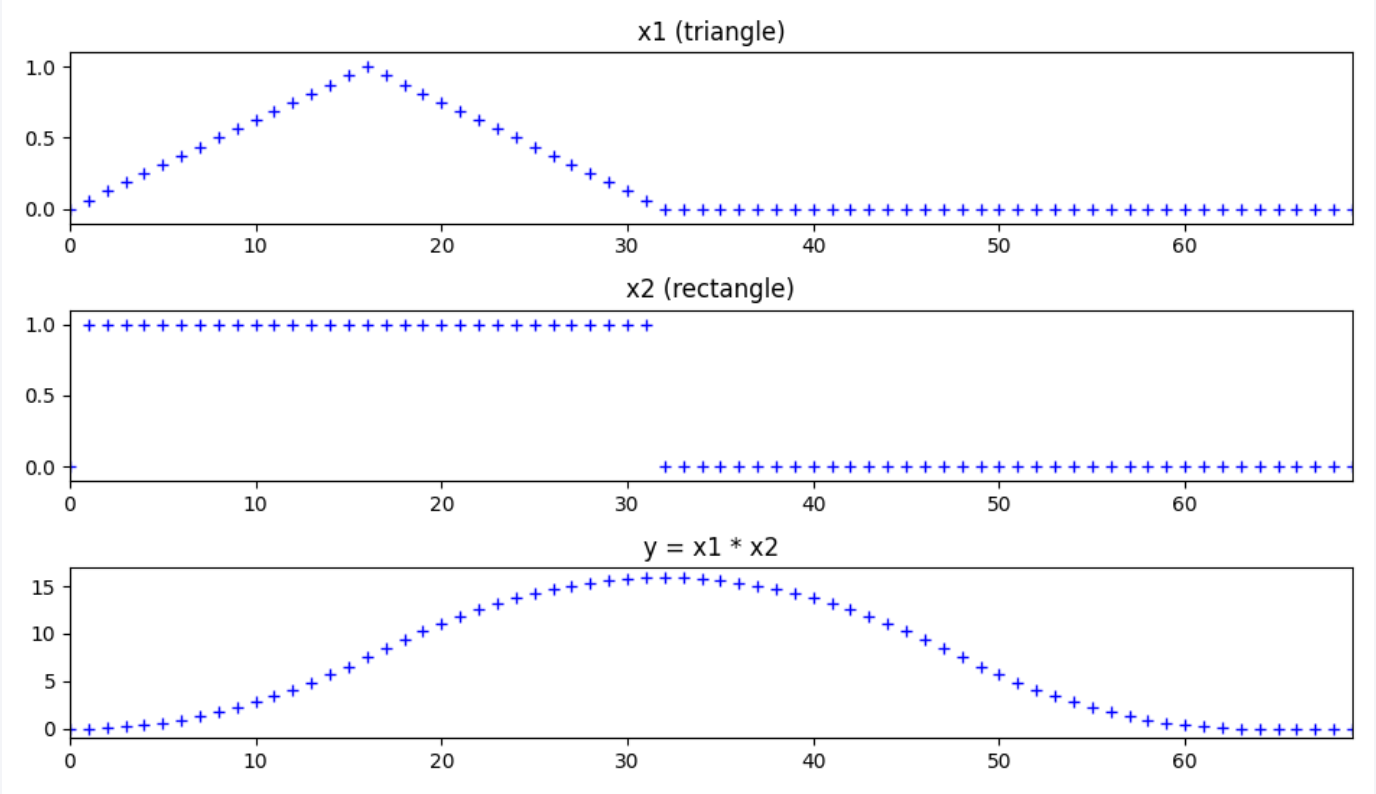
Any signal can be decomposed into an even part and an odd part. These symmetries are useful when interpreting Fourier-transform properties.



MATH

Convolution in discrete time

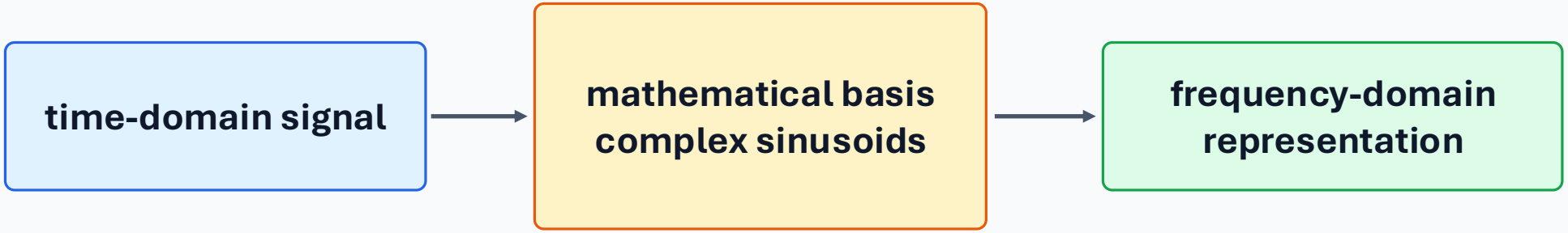
$$y[n] = (x_1[n] * x_2[n])_n = \sum_{m=0}^{N-1} x_1[m] x_2[n-m]$$



Convolution combines an input signal with a system response.

In LTI systems, it gives the output for any input.

The first big conceptual bridge



Fourier analysis is a change of viewpoint: from samples over time to weights of sinusoidal building blocks.

REFERENCES

Recommended references for Lecture 1

- MTG sms-tools-materials repository: lecture material, exercises, example notebooks, sounds, and GUI interfaces.
- Julius O. Smith, Mathematics of the DFT: a rigorous online reference for DFT foundations.
- Wikipedia background refreshers: audio signal processing, sinusoid, complex numbers, Euler formula, dot product, convolution.
- Audacity and sound examples can be used to listen to basic transformations and effects.

Keep references practical: read only what helps you implement, listen, and interpret the current week's material.

LAB

Exercise 1: Python and sounds

First lab: move from formulas to samples, code, plots, and listening.

What you will practice

- read a mono WAV file into a NumPy array
- inspect samples, indexing, amplitude range, and plots
- write small functions for reproducible audio operations
- downsample sounds and listen to the consequences

How to start

Open: `exercises/E1-Python-and-sounds.ipynb`

Use: NumPy · Matplotlib · IPython Audio · wavread()

Sounds: piano.wav · oboe-A4.wav · vibraphone-C6.wav · sawtooth-440.wav

Lab structure

1 · Reading audio

- read_audio()
- return sample slices

2 · Basic audio ops

- min_max_audio()
- plot and inspect waveform

3 · Array indexing

- hop_samples(x, M)
- select every Mth element

4 · Downsampling

- M = 14
- listen for aliasing

Listening question: what changes when we keep fewer samples, and why do different sounds react differently?

Digital Signal Processing for Sound and Music · Lecture 1 MTG / UPF